arXiv:2312.00858v2 [cs.CV] 7 Dec 2023

DeepCache: Accelerating Diffusion Models for Free

Xinyin Ma **Gongfan Fang** **Xinchao Wang***
National University of Singapore
{maxinyin, gongfan}@u.nus.edu, xinchao@nus.edu.sg

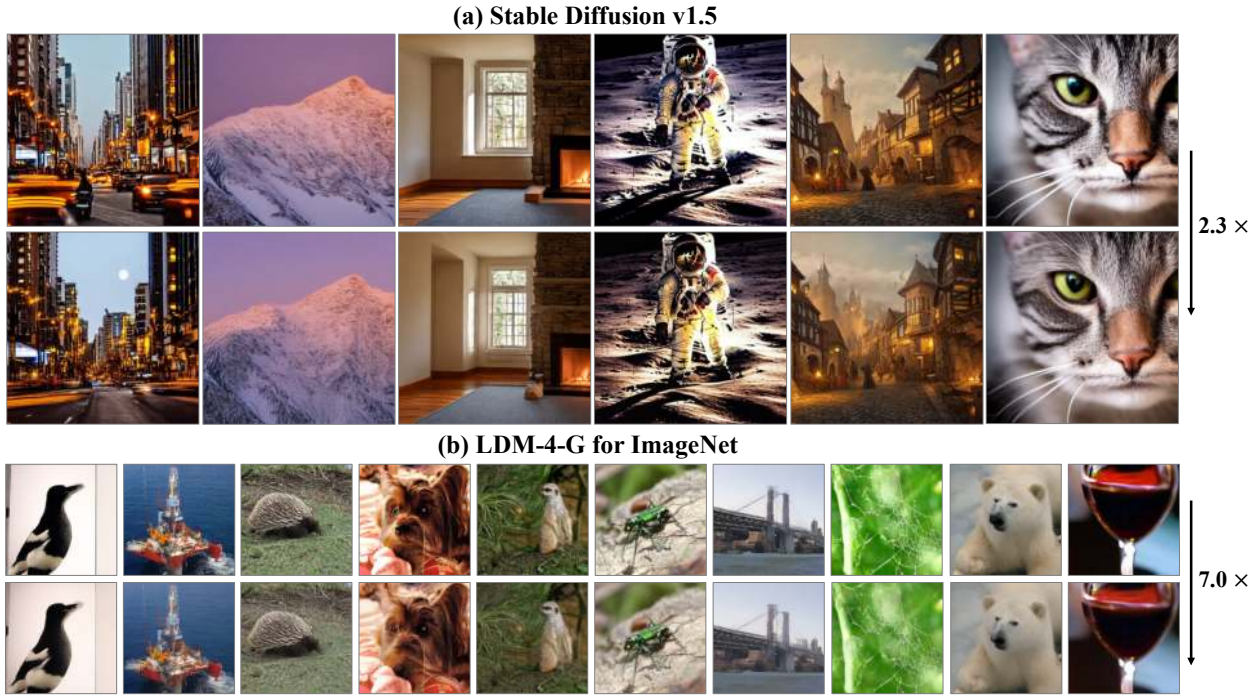

Figure 1. Accelerating Stable Diffusion V1.5 and LDM-4-G by 2.3× and 7.0×, with 50 PLMS steps and 250 DDIM steps respectively.

Abstract

Diffusion models have recently gained unprecedented attention in the field of image synthesis due to their remarkable generative capabilities. Notwithstanding their prowess, these models often incur substantial computational costs, primarily attributed to the sequential denoising process and cumbersome model size. Traditional methods for compressing diffusion models typically involve extensive retraining, presenting cost and feasibility challenges. In this paper, we introduce DeepCache, a novel training-free paradigm that accelerates diffusion models from the perspective of model architecture. DeepCache capitalizes on the inherent temporal redundancy observed in the sequential denoising steps of diffusion models, which caches and retrieves features across adjacent denoising stages, thereby curtailing redundant computations. Utilizing the property of the U-Net, we reuse the high-level features while updating the low-level features in a very cheap way. This innovative strategy, in turn, enables a speedup factor of 2.3× for Stable Diffusion v1.5 with only a 0.05 decline in CLIP Score, and 4.1× for LDM-4-G with a slight decrease of 0.22 in FID on ImageNet. Our experiments also demonstrate DeepCache's superiority over existing pruning and distillation methods that necessitate retraining and its compatibility with current sampling techniques. Furthermore, we find that under the same throughput, DeepCache effectively achieves comparable or even marginally improved results with DDIM or PLMS. Code is available at https://github.com/horseee/DeepCache.

1. Introduction

In recent years, diffusion models [9, 17, 57, 59] have emerged as a pivotal advancement in the field of genera-

* Corresponding author

tive modeling, gaining substantial attention for their impressive capabilities. These models have demonstrated remarkable efficacy across diverse applications, being employed for the generation of images [19, 60, 64], text [11, 28], audio [6, 44], and video [18, 36, 56]. A large number of attractive applications have been facilitated with diffusion models, including but not limited to image editing [2, 20, 38], image super-enhancing [26, 51], image-to-image translation [7, 49], text-to-image generation [41, 45, 46, 50] and text-to-3D generation [30, 35, 43].

Despite the significant effectiveness of diffusion models, their relatively slow inference speed remains a major obstacle to broader adoption, as highlighted in [29]. The core challenge stems from the step-by-step denoising process required during their reverse phase, limiting parallel decoding capabilities [55]. Efforts to accelerate these models have focused on two main strategies: reducing the number of sampling steps, as seen in approaches [34, 39, 52, 58], and decreasing the model inference overhead per step through methods like model pruning, distillation, and quantization [10, 13, 21].

Our goal is to enhance the efficiency of diffusion models by reducing model size at each step. Previous compression methods for diffusion models focused on re-designing network architectures through a comprehensive structural analysis [29] or involving frequency priors into the model design [66], which yields promising results on image generation. However, they require large-scale datasets for retraining these lightweight models. Pruning-based methods, as explored by [10, 21], lessen the data and training requirements to 0.1% of the full training. Alternatively, [32] employs adaptive models for different steps, which is also a potential solution. However, it depends on a collection of pre-trained models or requires optimization of subnetworks [65]. Those methods can reduce the expense of crafting a new lightweight model, but the retraining process is still inevitable, which makes the compression costly and less practical for large-scale pre-trained diffusion models, such as Stable Diffusion [47].

To this end, we focus on a challenging topic: *How to significantly reduce the computational overhead at each denoising step without additional training, thereby achieving a cost-free compression of Diffusion Models?* To achieve this, we turn our attention to the intrinsic characteristics of the reverse denoising process of diffusion models, observing a significant temporal consistency in the high-level features between consecutive steps. We found that those high-level features are even cacheable, which can be calculated once and retrieved again for the subsequent steps. By leveraging the structural property of U-Net, the high-level features can be cached while maintaining the low-level features updated at each denoising step. Through this, a considerable enhancement in the efficiency and speed of Diffusion Models can be achieved without any training.

To summarize, we introduce a novel paradigm for the acceleration of Diffusion Models, which gives a new perspective for training-free accelerating the diffusion models. It is not merely compatible with existing fast samplers but also shows potential for comparable or superior generation capabilities. The contributions of our paper include:

- We introduce a simple and effective acceleration algorithm, named DeepCache, for dynamically compressing diffusion models during runtime and thus enhancing image generation speed without additional training burdens.
- DeepCache utilizes the temporal consistency between high-level features. With the cacheable features, the redundant calculations are effectively reduced. Furthermore, we introduce a non-uniform 1:N strategy, specifically tailored for long caching intervals.
- DeepCache is validated across a variety of datasets, including CIFAR, LSUN-Bedroom/Churches, ImageNet, COCO2017 and PartiPrompt, and tested under DDPM, LDM, and Stable Diffusion. Experiments demonstrate that our approach has superior efficacy than pruning and distillation algorithms that require retraining under the same throughput.

2. Related Work

High-dimensional image generation has evolved significantly in generative modeling. Initially, GANs [1, 12] and VAEs [16, 22] led this field but faced scalability issues due to instability and mode collapse [23]. Recent advancements have been led by Diffusion Probabilistic Models [9, 17, 47, 61], which offer superior image generation. However, the inherent nature of the reverse diffusion process [59] slows down inference. Current research is focused on two main methods to speed up diffusion model inference.

Optimized Sampling Efficiency. focuses on reducing the number of sampling steps. DDIM [58] reduces these steps by exploring a non-Markovian process, related to neural ODEs. Studies [3, 33, 34, 69] further dive into the fast solver of SDE or ODE to create efficient sampling of diffusion models. Some methods progressively distilled the model to reduced timestep [52] or replace the remaining steps with a single-step VAE [37]. The Consistency Model [62] converts random noise to the initial images with only one model evaluation. Parallel sampling techniques like DSNO [70] and ParaDiGMS [55] employ Fourier neural operators and Picard iterations for parallel decoding .

Optimized Structural Efficiency. This approach aims to reduce inference time at each sampling step. It leverages strategies like structural pruning in Diff-pruning [10] and efficient structure evolving in SnapFusion [29]. Spectral

Diffusion [66] enhances architectural design by incorporating frequency dynamics and priors. In contrast to these methods, which use a uniform model at each step, [32] proposes utilizing different models at various steps, selected from a diffusion model zoo. The early stopping mechanism in diffusion is explored in [27, 40, 63], while the quantization techniques [13, 54] focus on low-precision data types for model weights and activations. Additionally, [4] and [5] present novel approaches to concentrate on inputs, with the former adopting a unique forward process for each pixel and the latter merging tokens based on similarity to enhance computational efficiency in attention modules. Our method is categorized under an objective to minimize the average inference time per step. Uniquely, our approach reduces the average model size substantially for each step, accelerating the denoising process without necessitating retraining.

3. Methodology

3.1. Preliminary

Forward and Reverse Process. Diffusion models [17] simulate an image generation process using a series of random diffusion steps. The core idea behind diffusion models is to start from random noise and gradually refine it until it resembles a sample from the target distribution. In the forward diffusion process, with a data point sampled from the real distribution, $\mathbf{x}_0 \sim q(\mathbf{x})$, gaussian noises are gradually added in T steps:

$$q\left(\mathbf{x}_t|\mathbf{x}_{t-1}\right) = \mathcal{N}\left(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t \mathbf{I}\right) \tag{1}$$

where t is the current timestep and $\{\beta_t\}$ schedules the noise. The *reverse diffusion process* denoises the random noise $x_T \sim \mathcal{N}(0, \mathbf{I})$ into the target distribution by modeling $q\left(\mathbf{x}_{t-1}|\mathbf{x}_t\right)$. At each reverse step t, the conditional probability distribution is approximated by a network $\epsilon_\theta\left(\mathbf{x}_t, t\right)$ with the timestep t and previous output $\mathbf{x}_t$ as input:

$$x_{t-1} \sim p_\theta(x_{t-1}|x_t) = \\ \mathcal{N}\left(x_{t-1}; \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta\left(\mathbf{x}_t, t\right)\right), \beta_t \mathbf{I}\right) \tag{2}$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^{T} \alpha_i$. Applied iteratively, it gradually reduces the noise of the current $\mathbf{x}_t$, bringing it close to a real data point when we reach x_0.

High-level and Low-level Features in U-Net. U-Net [48] was originally introduced for biomedical image segmentation and showcased a strong ability to amalgamate low-level and high-level features, attributed to the skip connections. U-Net is constructed on stacked downsampling and upsampling blocks, which encode the input image into a high-level representation and then decode it for downstream tasks. The block pairs, denoted as $\{D_i\}_{i=1}^{d}$ and $\{U_i\}_{i=1}^{d}$, are interconnected with additional skip paths. Those skip paths directly forward the rich and relatively more low-level information from D_i to U_i. During the forward propagation in the U-Net architecture, the data traverses concurrently through two pathways: the *main branch* and the *skip branch*. These branches converge at a concatenation module, with the *main branch* providing processed high-level feature from the preceding upsampling block U_{i+1}, and the *skip branch* contributing corresponding feature from the symmetrical block D_i. Therefore, at the heart of a U-Net model is a concatenation of low-level features from the skip branch, and the high-level features from the main branch, formalized as:

$$\text{Concat}(D_i(\cdot), U_{i+1}(\cdot)) \tag{3}$$

3.2. Feature Redundancy in Sequential Denoising

The inherent sequentiality of the denoising process in diffusion models presents a primary bottleneck in inference speed. Previous methods primarily employed strategies that involved skipping certain steps to address this issue. In this work, we revisit the entire denoising process, seeking to uncover specific properties that could be optimized to enhance inference efficiency.

Observation. *Adjacent steps in the denoising process exhibit significant temporal similarity in high-level features.*

In Figure 2, we provide empirical evidence related to this observation. The experiments elucidate two primary insights: 1) There is a noticeable temporal feature similarity between adjacent steps within the denoising process, indicating that the change between consecutive steps is typically minor; 2) Regardless of the diffusion model we used, for each timestep, at least 10% of the adjacent timesteps exhibit a high similarity (>0.95) to the current step, suggesting that certain high-level features change at a gradual pace. This phenomenon can be observed in a large number of well-established models like Stable Diffusion, LDM, and DDPM. In the case of DDPM for LSUN-Churches and LSUN-Bedroom, some timesteps even demonstrate a high degree of similarity to 80% of the other steps, as highlighted in the green line in Figure 2 (c).

Building upon these observations, our objective is to leverage this advantageous characteristic to accelerate the denoising process. Our analysis reveals that the computation often results in a feature remarkably similar to that of the previous step, thereby highlighting the existence of redundant calculations for optimization. We contend that allocating significant computational resources to regenerate these analogous feature maps constitutes an inefficiency. While incurring substantial computational expense, yields marginal benefits, it suggests a potential area for efficiency improvements in the speed of diffusion models.

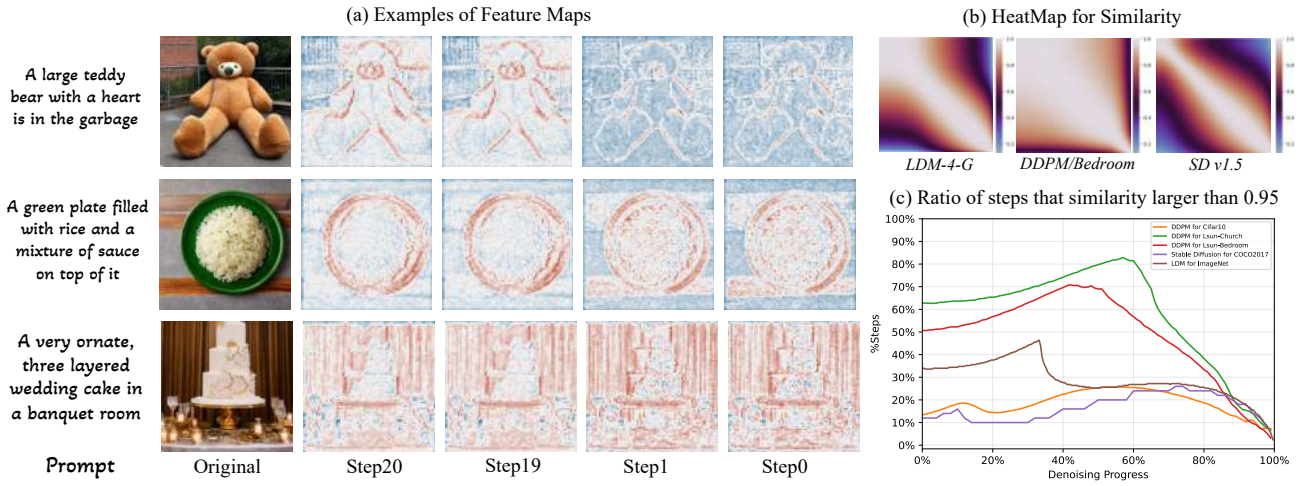

Figure 2. (a) Examples of feature maps in the up-sampling block U_2 in Stable Diffusion. We present a comparison from two adjacently paired steps, emphasizing the invariance inherent in the denoising process. (b) Heatmap of similarity between U_2's features in all steps on three typical diffusion models. (c) The percentage of steps with a similarity greater than 0.95 to the current step.

3.3. Deep Cache For Diffusion Models

We introduce DeepCache, a simple and effective approach that leverages the temporal redundancy between steps in the reverse diffusion process to accelerate inference. Our method draws inspiration from the caching mechanism in a computer system, incorporating a storage component designed for elements that exhibit minimal changes over time. Applying this in diffusion models, we eliminate redundant computations by strategically caching slowly evolving features, thereby obviating the need for repetitive recalculations in subsequent steps.

To achieve this, we shift our focus to the skip connections within U-Net, which inherently offers a dual-pathway advantage: the main branch requires heavy computation to traverse the entire network, while the skip branch only needs to go through some shallow layers, resulting in a very small computational load. The prominent feature similarity in the main branch, allows us to reuse the already computed results rather than calculate it repeatedly for all timesteps.

Cacheable Features in denosing. To make this idea more concrete, we study the case within two consecutive timesteps t and $t-1$. According to the reverse process, x_{t-1} would be conditionally generated based on the previous results x_t. First, we generate x_t in the same way as usual, where the calculations are performed across the entire U-Net. To obtain the next output x_{t-1}, we retrieve the high-level features produced in the previous x_t. More specifically, consider a skip branch m in the U-Net, which bridges D_m and U_m, we cache the feature maps from the previous up-sampling block at the time t as the following:

$$F^t_{\text{cache}} \leftarrow U^t_{m+1}(\cdot) \tag{4}$$

which is the feature from the main branch at timestep t. Those cached features will be plugged into the network inference in the subsequent steps. In the next timestep $t-1$, the inference is not carried out on the entire network; instead, we perform a dynamic partial inference. Based on the previously generated x_t, we only calculate those that are necessary for the m-th skip branch and substitute the compute of the main branch with a retrieving operation from the cache in Equation 4. Therefore, the input for U^{t-1}_m in the $t-1$ timestep can be formulated as:

$$\text{Concat}(D^{t-1}_m(\cdot), F^t_{\text{cache}}) \tag{5}$$

Here, D^{t-1}_m represents the output of the m-th downsampling block, which only contains a few layers if a small m is selected. For example, if we perform DeepCache at the first layer with $m = 1$, then we only need to execute one downsampling block to obtain D^{t-1}_1. As for the second feature F^t_{cache}, no additional computational cost is needed since it can be simply retrieved from the cache. We illustrate the above process in Figure 3.

Extending to 1:N Inference This process is not limited to the type with one step of full inference followed by one step of partial inference. As shown in Figure 2(b), pairwise similarity remains a high value in several consecutive steps. The mechanism can be extended to cover more steps, with the cached features calculated once and reused in the consecutive $N-1$ steps to replace the original $U^{t-n}_{m+1}(\cdot)$, $n \in \{1, \ldots, N-1\}$. Thus, for all the T steps for denoising, the sequence of timesteps that performs full inference are:

$$\mathcal{I} = \{x \in \mathbb{N} \,|\, x = iN,\ 0 \le i < k\} \tag{6}$$

where $k = \lceil T/N \rceil$ denotes the times for cache updating.

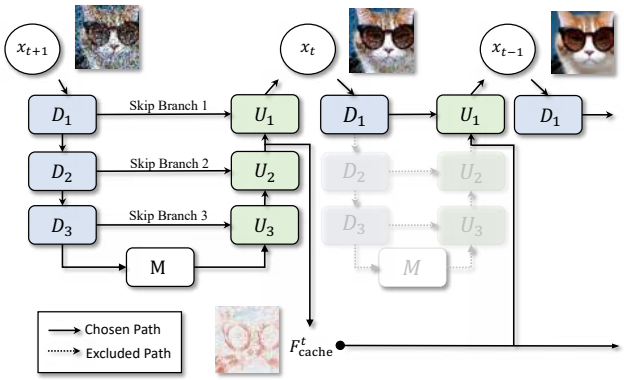

Figure 3. Illustration of DeepCache. At the $t-1$ step, x_{t-1} is generated by reusing the features cached at the t step, and the blocks D_2, D_3, U_2, U_3 are not executed for more efficient inference.

Non-uniform 1:N Inference Based on the 1:N strategy, we managed to accelerate the inference of diffusion under a strong assumption that the high-level features are invariant in the consecutive N step. However, it's not invariably the case, especially for a large N, as demonstrated by the experimental evidence in Figure 2(c). The similarity of the features does not remain constant across all steps. For models such as LDM, the temporal similarity of features would significantly decrease around 40% of the denoising process. Thus, for the non-uniform 1:N inference, we tend to sample more on those steps with relatively small similarities to the adjacent steps. Here, the sequence of timesteps to perform full inference becomes:

$$\mathcal{L} = \left\{ l_i \mid l_i \in \text{linear_space}\left((-c)^{\frac{1}{p}}, (T-c)^{\frac{1}{p}}, k \right) \right\} \quad (7)$$
$$\mathcal{I} = \text{unique_int}\left(\{ i_k \mid i_k = (l_k)^p + c, \text{ where } l_k \in \mathcal{L} \} \right)$$

where $\text{linear_space}(s, e, n)$ evenly spaces n numbers from s to e (exclusive) and $\text{unique_int}(\cdot)$ convert the number to int and ensure the uniqueness of timesteps in the sequence. c is the hyper-parameter for the selected center timestep. In this equation, the frequency of indexes changes in a quadratic manner as it moves away from a central timestep. It is essential to note that the aforementioned strategy represents one among several potential strategies. Alternative sequences, particularly centered on a specific timestep, can also yield similar improvements in image quality.

4. Experiment

4.1. Experimental Settings

Models, Datasets and Evaluation Metrics To demonstrate the effectiveness of our method is agnostic with the type of pre-trained DMs, we evaluate our methods on three commonly used DMs: DDPM [17], LDM [47] and Stable Diffusion [47][1]. Except for this, to show that our method is compatible with the fast sampling methods, we build our method upon 100-step DDIM [58] for DDPM, 250-step for LDM and 50-step PLMS [33] for Stable Diffusion, instead of the complete 1000-step denoising process. We select six datasets that are commonly adopted to evaluate these models, including CIFAR10 [24], LSUN-Bedroom [67], LSUN-Churches [67], ImageNet [8], MS-COCO 2017 [31] and PartiPrompts [68]. For MS-COCO 2017 and PartiPrompt, we utilized the 5k validation set and 1.63k captions respectively as prompts for Stable Diffusion. For other datasets, we generate 50k images to assess the generation quality. We follow previous works [10, 55, 66] to employ the evaluation metrics including FID, sFID, IS, Precision-and-Recall and CLIP Score (on ViT-g/14) [14, 15, 25, 53].

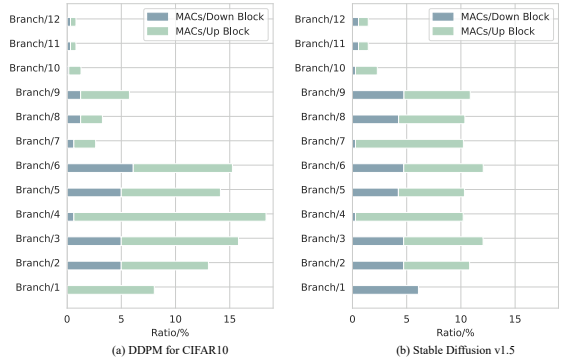

Figure 4. MACs for each skip branch, evaluated on DDPM for CIFAR10 and Stable Diffusion V1.5.

Baselines We choose Diff-Pruning [10] as the main baseline for our method since Diff-Pruning also reduces the training effort for compressing DMs. For the experiment on LDM, we extend [66] as another baseline to represent one of the best methods to re-design a lightweight diffusion model. For the experiment on Stable Diffusion, we choose BK-SDMs [21], which are trained on 2.3M LAION image-text pairs, as baselines of architecturally compression and distillation for Stable Diffusion.

4.2. Complexity Analysis

We first analyze the improvement in inference speed facilitated by DeepCache. The notable acceleration in inference speed primarily arises from incomplete reasoning in denoising steps, with layer removal accomplished by partitioning the U-Net by the skip connection. In Figure 4, we present the division of MACs on two models. For each skip branch i, the MACs here contain the MACs in down block D_i and the up block U_i. There is a difference in the amount

[1]https://huggingface.co/runwayml/stable-diffusion-v1-5

ImageNet 256 × 256									
Method	**MACs ↓**	**Throughput ↑**	**Speed ↑**	**Retrain**	**FID ↓**	**sFID ↓**	**IS ↑**	**Precision ↑**	**Recall ↑**
IDDPM [42]	1416.3G	-	-	✗	12.26	5.42	-	70.0	62.0
ADM-G [9]	1186.4G	-	-	✗	4.59	5.25	186.70	82.0	52.0
LDM-4 [47]	99.82G	0.178	1×	✗	3.60	-	247.67	87.0	48.0
LDM-4*	99.82G	0.178	1×	✗	3.37	5.14	204.56	82.71	53.86
Spectral DPM [66]	9.9G	-	-	✓	10.60	-	-	-	-
Diff-Pruning [10]*	52.71G	0.269	1.51×	✓	$9.27_{(9.16)}$	10.59	$214.42_{(201.81)}$	87.87	30.87
Uniform - N=2	52.12G	0.334	1.88×	✗	3.39	5.11	204.09	82.75	54.07
Uniform - N=3	36.48G	0.471	2.65×	✗	3.44	5.11	202.79	82.65	53.81
Uniform - N=5	23.50G	0.733	4.12×	✗	3.59	5.16	200.45	82.36	53.31
Uniform - N=10	13.97G	1.239	6.96×	✗	4.41	5.57	191.11	81.26	51.53
Uniform - N=20	9.39G	1.876	10.54×	✗	8.23	8.08	161.83	75.31	50.57
NonUniform - N=10	13.97G	1.239	6.96×	✗	4.27	5.42	193.11	81.75	51.84
NonUniform - N=20	9.39G	1.876	10.54×	✗	7.11	7.34	167.85	77.44	50.08

Table 1. Class-conditional generation quality on ImageNet using LDM-4-G. The baselines here, as well as our methods, employ 250 DDIM steps. *We reproduce Diff-Pruning to have a comprehensive comparison and the official results are shown in brackets.

CIFAR-10 32 × 32					
Method	**MACs ↓**	**Throughput ↑**	**Speed ↑**	**Steps ↓**	**FID ↓**
DDPM	6.1G	9.79	1×	-	4.19
DDPM*	6.1G	9.79	1×	-	4.16
Diff-Pruning	3.4G	13.45	1.37×	100k	5.29
Ours - N=2	4.15G	13.73	1.40×	0	4.35
Ours - N=3	3.54G	15.74	1.61×	0	4.70
Ours - N=5	3.01G	18.11	1.85×	0	5.73
Ours - N=10	2.63G	20.26	2.07×	0	9.74
LSUN-Bedroom 256 × 256					
Method	**MACs ↓**	**Throughput ↑**	**Speed ↑**	**Steps ↓**	**FID ↓**
DDPM	248.7G	0.21	1×	-	6.62
DDPM*	248.7G	0.21	1×	-	6.70
Diff-Pruning	138.8G	0.31	1.48×	200k	18.60
Ours - N=2	190.8G	0.27	1.29×	0	6.69
Ours - N=3	172.3G	0.30	1.43×	0	7.20
Ours - N=5	156.0G	0.31	1.48×	0	9.49
LSUN-Churches 256 × 256					
Method	**MACs ↓**	**Throughput ↑**	**Speed ↑**	**Steps ↓**	**FID ↓**
DDPM	248.7G	0.21	1×	-	10.58
DDPM*	248.7G	0.21	1×	-	10.87
Diff-Pruning	138.8G	0.31	1.48×	500k	13.90
Ours - N=2	190.8G	0.27	1.29×	0	11.31
Ours - N=3	172.3G	0.30	1.43×	0	11.75
Ours - N=5	156.0G	0.31	1.48×	0	13.68

Table 2. Results on CIFAR-10, LSUN-Bedroom and LSUN-Churches. All the methods here adopt 100 DDIM steps. * means the reproduced results, which are more comparable with our results since the random seed is the same.

of computation allocated to different skip branches for different models. Stable diffusion demonstrates a comparatively uniform distribution across layers, whereas DDPM exhibits more computational burden concentrated within the first several layers. Our approach would benefit from U-Net structures that have a larger number of skip branches, facilitating finer divisions of models, and giving us more choices for trade-off the speed and quality. In our experiment, we choose the skip branch 3/1/2 for DDPMs, LDM-4-G and Stable Diffusion respectively. We provide the results of using different branches in Appendix.

To comprehensively evaluate the efficiency of our method, in the following experiments, we report the throughput of each model using a single RTX2080 GPU. Besides, we report MACs in those tables, which refer to the average MACs for all steps.

4.3. Comparison with Compression Methods

LDM-4-G for ImageNet. We conduct experiments on ImageNet, and the results are shown in Table 1. When accelerating to 4.1× the speed, a minor performance decline is observed (from 3.39 to 3.59). Compared with the pruning and distillation methods, a notable improvement over those methods is observed in FID and sFID, even though the acceleration ratio of our method is more substantial. Furthermore, the augmentation in quality becomes more obvious with a larger number N of caching intervals if we employ the non-uniform 1:N strategy. Detailed results for the non-uniform 1:N strategy for small N and the hyper-parameters for the non-uniform strategy are provided in the Appendix.

DDPMs for CIFAR-10 and LSUN. The results on CIFAR10, LSUN-Bedroom and LSUN-Churches are shown in Table 2. From these tables, we can find out that our method surpasses those requiring retraining, even though our methods have no retraining cost. Additionally, since we adopt a layer-pruning approach, which is more hardware-friendly, our acceleration ratio is more significant compared to the baseline method, under similar MACs constraints.

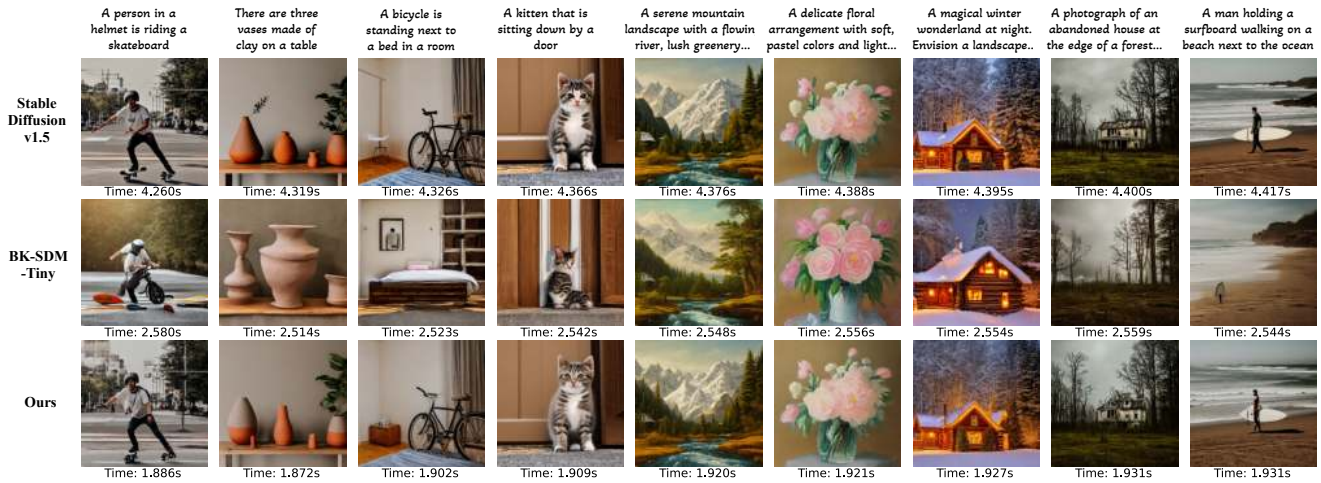

Figure 5. Visualization of the generated images by BK-SDM-Tiny and DeepCache. All the methods adopt the 50-step PLMS. The time here is the duration to generate a single image. Some prompts are omitted from this section for brevity. See Appendix for details.

	PartiPrompts				COCO2017			
Method	**MACs** ↓	**Throughput** ↑	**Speed** ↑	**CLIP Score** ↑	**MACs** ↓	**Throughput** ↑	**Speed** ↑	**CLIP Score** ↑
PLMS - 50 steps	338.83G	0.230	1.00×	29.51	338.83G	0.237	1.00×	30.30
PLMS - 25 steps	169.42G	0.470	2.04×	29.33	169.42G	0.453	1.91×	29.99
BK-SDM - Base	223.81G	0.343	1.49×	28.88	223.81G	0.344	1.45 ×	29.47
BK-SDM - Small	217.78G	0.402	1.75×	27.94	217.78G	0.397	1.68×	27.96
BK-SDM - Tiny	205.09G	0.416	1.81×	27.36	205.09G	0.415	1.76 ×	27.41
Ours	130.45G	0.494	2.15×	29.46	130.45G	0.500	2.11×	30.23

Table 3. Comparison with PLMS and BK-SDM. We utilized prompts in PartiPrompt and COCO2017 validation set to generate images at the resolution of 512. We choose N=5 to achieve a throughput that is comparable to or surpasses that of established baseline methods. Results for other choices of N can be found in Figure 6.

Stable Diffusion. The results are presented in Table 3. We outperform all three variants of BK-SDM, even with a faster denoising speed. As evident from the showcased examples in Figure 5, the images generated by our method exhibit a greater consistency with the images generated by the original diffusion model, and the image aligns better with the textual prompt.

4.4. Comparison with Fast Sampler.

We conducted a comparative analysis with methods focused on reducing sampling steps. It is essential to highlight that our approach is additive to those fast samplers, as we show in previous experiments. In Table 3 and Table 4, we first compared our method with the DDIM [58] or PLMS [33] under similar throughputs. We observe that our method achieved slightly better results than 25-step PLMS on Stable Diffusion and comparable results to DDIM on LDM-4-G. We also measured the performance comparison between PLMS and our method under different acceleration ratios in Figure 6 to provide a more comprehensive comparison.

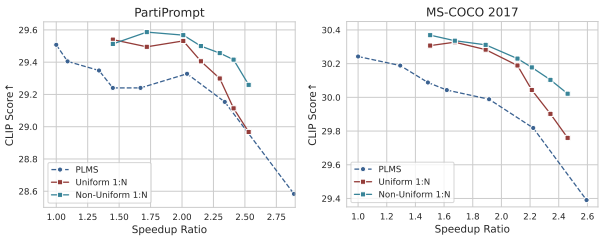

Figure 6. Comparison between PLMS, DeepCache with uniform 1:N and non-uniform 1:N stratigies.

4.5. Analysis

Ablation Study. DeepCache can be conceptualized as incorporating $(N-1)\times K$ steps of shallow network inference on top of the DDIM's K steps, along with more updates of the noisy images. It is important to validate whether the additional computations of shallow network inference and the caching of features yield positive effectiveness: 1) **Effectiveness of Cached Features:** We assess the impact of

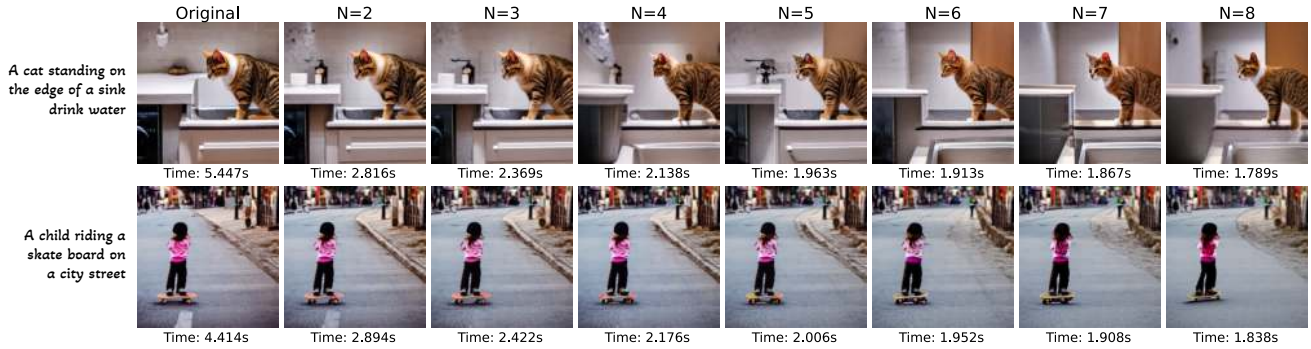

Figure 7. Illustration of the evolution in generated images with increasing caching interval N.

Method	Throughput ↑	FID ↓	sFID ↓
DDIM - 59 steps	0.727	**3.59**	**5.14**
Ours	0.733	**3.59**	5.16
DDIM - 91 steps	0.436	3.46	**50.6**
Ours	0.471	**3.44**	5.11

Table 4. Comparison with DDIM under the same throughput. Here we conduct class-conditional for ImageNet using LDM-4-G.

cached features in Table 5. Remarkably, we observe that, without any retraining, the cached features play a pivotal role in the effective denoising of diffusion models employing a shallow U-Net. 2) **Positive Impact of Shallow Network Inference:** Building upon the cached features, the shallow network inference we conduct has a positive impact compared to DDIM. Results presented in Table 6 indicate that, with the additional computation of the shallow U-Net, DeepCache improves the 50-step DDIM by 0.32 and the 10-step DDIM by 2.98.

Illustration of the increasing caching interval N. In Figure 7, we illustrate the evolution of generated images as we increment the caching interval. A discernible trend emerges as a gradual reduction in time, revealing that the primary features of the images remain consistent with their predecessors. However, subtle details such as the color of clothing and the shape of the cat undergo modifications. Quantitative insights are provided in Table 1 and Figure 6, where with an interval $N < 5$, there is only a slight reduction in the quality of the generated image.

5. Limitations

The primary limitation of our method originates from its dependence on the pre-defined structure of the pre-trained diffusion model. Specifically, when a model's shallowest skip branch encompasses a substantial portion of computation, such as 50% of the whole model, the achievable speedup ratio through our approach becomes relatively constrained. Additionally, our method encounters non-negligible performance degradation with larger caching steps (e.g., N=20), which could impose constraints on the upper limit of the acceleration ratio.

Model	Dataset	DeepCache	w/o Cached Features
DDPM	Cifar10	9.74	192.98
LDM-4-G	ImageNet	7.36	312.12

Table 5. Effectiveness of Cached Features. Under identical hyper-parameters, we replace the cached features with a zero matrix.

Steps	DDIM FID↓	DeepCache FID↓	Δ
50	4.67	4.35	-0.32
20	6.84	5.73	-1.11
10	13.36	10.38	-2.98

Table 6. Effectiveness of Shallow Network Inference. Steps here mean the number of steps that perform full model inference.

6. Conclusion

In this paper, we introduce a novel paradigm, DeepCache, to accelerate the diffusion model. Our strategy employs the similarity observed in high-level features across adjacent steps of the diffusion model, thereby mitigating the computational overhead associated with redundant high-level feature calculations. Additionally, we leverage the structural attributes in the U-Net architecture to facilitate the updating of low-level features. Through the adoption of DeepCache, a noteworthy acceleration in computational speed is achieved. Empirical evaluations on several datasets and diffusion models demonstrate that DeepCache surpass other compression methods that focus on the reduction of parameter size. Moreover, the proposed algorithm demonstrates comparable and even slightly superior generation quality compared to existing techniques such as DDIM and PLMS, thereby offering a new perspective in the field.

References

[1] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *International conference on machine learning*, pages 214–223. PMLR, 2017. 2

[2] Omri Avrahami, Dani Lischinski, and Ohad Fried. Blended diffusion for text-driven editing of natural images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18208–18218, 2022. 2

[3] Fan Bao, Chongxuan Li, Jun Zhu, and Bo Zhang. Analytic-dpm: an analytic estimate of the optimal reverse variance in diffusion probabilistic models. *arXiv preprint arXiv:2201.06503*, 2022. 2

[4] Georgios Batzolis, Jan Stanczuk, Carola-Bibiane Schönlieb, and Christian Etmann. Non-uniform diffusion models. *arXiv preprint arXiv:2207.09786*, 2022. 3

[5] Daniel Bolya and Judy Hoffman. Token merging for fast stable diffusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4598–4602, 2023. 3

[6] Nanxin Chen, Yu Zhang, Heiga Zen, Ron J Weiss, Mohammad Norouzi, and William Chan. Wavegrad: Estimating gradients for waveform generation. *arXiv preprint arXiv:2009.00713*, 2020. 2

[7] Jooyoung Choi, Sungwon Kim, Yonghyun Jeong, Youngjune Gwon, and Sungroh Yoon. Ilvr: Conditioning method for denoising diffusion probabilistic models. *arXiv preprint arXiv:2108.02938*, 2021. 2

[8] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 5

[9] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems*, 34:8780–8794, 2021. 1, 2, 6

[10] Gongfan Fang, Xinyin Ma, and Xinchao Wang. Structural pruning for diffusion models. In *Advances in Neural Information Processing Systems*, 2023. 2, 5, 6

[11] Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and LingPeng Kong. Diffuseq: Sequence to sequence text generation with diffusion models. *arXiv preprint arXiv:2210.08933*, 2022. 2

[12] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014. 2

[13] Yefei He, Luping Liu, Jing Liu, Weijia Wu, Hong Zhou, and Bohan Zhuang. Ptqd: Accurate post-training quantization for diffusion models. *arXiv preprint arXiv:2305.10657*, 2023. 2, 3

[14] Jack Hessel, Ari Holtzman, Maxwell Forbes, Ronan Le Bras, and Yejin Choi. Clipscore: A reference-free evaluation metric for image captioning. *arXiv preprint arXiv:2104.08718*, 2021. 5

[15] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30, 2017. 5

[16] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. beta-vae: Learning basic visual concepts with a constrained variational framework. In *International conference on learning representations*, 2016. 2

[17] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020. 1, 2, 3, 5, 12

[18] Jonathan Ho, William Chan, Chitwan Saharia, Jay Whang, Ruiqi Gao, Alexey Gritsenko, Diederik P Kingma, Ben Poole, Mohammad Norouzi, David J Fleet, et al. Imagen video: High definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*, 2022. 2

[19] Bahjat Kawar, Michael Elad, Stefano Ermon, and Jiaming Song. Denoising diffusion restoration models. *Advances in Neural Information Processing Systems*, 35:23593–23606, 2022. 2

[20] Bahjat Kawar, Shiran Zada, Oran Lang, Omer Tov, Huiwen Chang, Tali Dekel, Inbar Mosseri, and Michal Irani. Imagic: Text-based real image editing with diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6007–6017, 2023. 2

[21] Bo-Kyeong Kim, Hyoung-Kyu Song, Thibault Castells, and Shinkook Choi. On architectural compression of text-to-image diffusion models. *arXiv preprint arXiv:2305.15798*, 2023. 2, 5

[22] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013. 2

[23] Naveen Kodali, Jacob Abernethy, James Hays, and Zsolt Kira. On convergence and stability of gans. *arXiv preprint arXiv:1705.07215*, 2017. 2

[24] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009. 5

[25] Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Improved precision and recall metric for assessing generative models. *Advances in Neural Information Processing Systems*, 32, 2019. 5

[26] Haoying Li, Yifan Yang, Meng Chang, Shiqi Chen, Huajun Feng, Zhihai Xu, Qi Li, and Yueting Chen. Srdiff: Single image super-resolution with diffusion probabilistic models. *Neurocomputing*, 479:47–59, 2022. 2

[27] Lijiang Li, Huixia Li, Xiawu Zheng, Jie Wu, Xuefeng Xiao, Rui Wang, Min Zheng, Xin Pan, Fei Chao, and Rongrong Ji. Autodiffusion: Training-free optimization of time steps and architectures for automated diffusion model acceleration. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7105–7114, 2023. 3

[28] Xiang Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. Diffusion-lm improves controllable text generation. *Advances in Neural Information Processing Systems*, 35:4328–4343, 2022. 2

[29] Yanyu Li, Huan Wang, Qing Jin, Ju Hu, Pavlo Chemerys, Yun Fu, Yanzhi Wang, Sergey Tulyakov, and Jian Ren. Snapfusion: Text-to-image diffusion model on mobile devices

within two seconds. *arXiv preprint arXiv:2306.00980*, 2023. 2

[30] Chen-Hsuan Lin, Jun Gao, Luming Tang, Towaki Takikawa, Xiaohui Zeng, Xun Huang, Karsten Kreis, Sanja Fidler, Ming-Yu Liu, and Tsung-Yi Lin. Magic3d: High-resolution text-to-3d content creation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 300–309, 2023. 2

[31] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014. 5

[32] Enshu Liu, Xuefei Ning, Zinan Lin, Huazhong Yang, and Yu Wang. Oms-dpm: Optimizing the model schedule for diffusion probabilistic models. *arXiv preprint arXiv:2306.08860*, 2023. 2, 3

[33] Luping Liu, Yi Ren, Zhijie Lin, and Zhou Zhao. Pseudo numerical methods for diffusion models on manifolds. *arXiv preprint arXiv:2202.09778*, 2022. 2, 5, 7

[34] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps. *Advances in Neural Information Processing Systems*, 35:5775–5787, 2022. 2

[35] Shitong Luo and Wei Hu. Diffusion probabilistic models for 3d point cloud generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2837–2845, 2021. 2

[36] Zhengxiong Luo, Dayou Chen, Yingya Zhang, Yan Huang, Liang Wang, Yujun Shen, Deli Zhao, Jingren Zhou, and Tieniu Tan. Videofusion: Decomposed diffusion models for high-quality video generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10209–10218, 2023. 2

[37] Zhaoyang Lyu, Xudong Xu, Ceyuan Yang, Dahua Lin, and Bo Dai. Accelerating diffusion models via early stop of the diffusion process. *arXiv preprint arXiv:2205.12524*, 2022. 2

[38] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. Sdedit: Guided image synthesis and editing with stochastic differential equations. *arXiv preprint arXiv:2108.01073*, 2021. 2

[39] Chenlin Meng, Robin Rombach, Ruiqi Gao, Diederik Kingma, Stefano Ermon, Jonathan Ho, and Tim Salimans. On distillation of guided diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14297–14306, 2023. 2

[40] Taehong Moon, Moonseok Choi, EungGu Yun, Jongmin Yoon, Gayoung Lee, and Juho Lee. Early exiting for accelerated inference in diffusion models. In *ICML 2023 Workshop on Structured Probabilistic Inference {\&} Generative Modeling*, 2023. 3

[41] Alex Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob McGrew, Ilya Sutskever, and Mark Chen. Glide: Towards photorealistic image generation and editing with text-guided diffusion models. *arXiv preprint arXiv:2112.10741*, 2021. 2

[42] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning*, pages 8162–8171. PMLR, 2021. 6

[43] Ben Poole, Ajay Jain, Jonathan T Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. *arXiv preprint arXiv:2209.14988*, 2022. 2

[44] Vadim Popov, Ivan Vovk, Vladimir Gogoryan, Tasnima Sadekova, and Mikhail Kudinov. Grad-tts: A diffusion probabilistic model for text-to-speech. In *International Conference on Machine Learning*, pages 8599–8608. PMLR, 2021. 2

[45] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 1 (2):3, 2022. 2

[46] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022. 2

[47] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022. 2, 5, 6

[48] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18*, pages 234–241. Springer, 2015. 3

[49] Chitwan Saharia, William Chan, Huiwen Chang, Chris Lee, Jonathan Ho, Tim Salimans, David Fleet, and Mohammad Norouzi. Palette: Image-to-image diffusion models. In *ACM SIGGRAPH 2022 Conference Proceedings*, pages 1–10, 2022. 2

[50] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily L Denton, Kamyar Ghasemipour, Raphael Gontijo Lopes, Burcu Karagol Ayan, Tim Salimans, et al. Photorealistic text-to-image diffusion models with deep language understanding. *Advances in Neural Information Processing Systems*, 35:36479–36494, 2022. 2

[51] Chitwan Saharia, Jonathan Ho, William Chan, Tim Salimans, David J Fleet, and Mohammad Norouzi. Image super-resolution via iterative refinement. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(4):4713–4726, 2022. 2

[52] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. *arXiv preprint arXiv:2202.00512*, 2022. 2

[53] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training gans. *Advances in neural information processing systems*, 29, 2016. 5

[54] Yuzhang Shang, Zhihang Yuan, Bin Xie, Bingzhe Wu, and Yan Yan. Post-training quantization on diffusion models. In

Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 1972–1981, 2023. 3

[55] Andy Shih, Suneel Belkhale, Stefano Ermon, Dorsa Sadigh, and Nima Anari. Parallel sampling of diffusion models. *arXiv preprint arXiv:2305.16317*, 2023. 2, 5

[56] Uriel Singer, Adam Polyak, Thomas Hayes, Xi Yin, Jie An, Songyang Zhang, Qiyuan Hu, Harry Yang, Oron Ashual, Oran Gafni, et al. Make-a-video: Text-to-video generation without text-video data. *arXiv preprint arXiv:2209.14792*, 2022. 2

[57] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. PMLR, 2015. 1

[58] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020. 2, 5, 7

[59] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019. 1, 2

[60] Yang Song, Sahaj Garg, Jiaxin Shi, and Stefano Ermon. Sliced score matching: A scalable approach to density and score estimation. In *Uncertainty in Artificial Intelligence*, pages 574–584. PMLR, 2020. 2

[61] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020. 2

[62] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. 2023. 2

[63] Shengkun Tang, Yaqing Wang, Caiwen Ding, Yi Liang, Yao Li, and Dongkuan Xu. Deediff: Dynamic uncertainty-aware early exiting for accelerating diffusion model generation. *arXiv preprint arXiv:2309.17074*, 2023. 3

[64] Arash Vahdat, Karsten Kreis, and Jan Kautz. Score-based generative modeling in latent space. *Advances in Neural Information Processing Systems*, 34:11287–11302, 2021. 2

[65] Shuai Yang, Yukang Chen, Luozhou Wang, Shu Liu, and Yingcong Chen. Denoising diffusion step-aware models. *arXiv preprint arXiv:2310.03337*, 2023. 2

[66] Xingyi Yang, Daquan Zhou, Jiashi Feng, and Xinchao Wang. Diffusion probabilistic model made slim. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22552–22562, 2023. 2, 3, 5, 6

[67] Fisher Yu, Ari Seff, Yinda Zhang, Shuran Song, Thomas Funkhouser, and Jianxiong Xiao. Lsun: Construction of a large-scale image dataset using deep learning with humans in the loop. *arXiv preprint arXiv:1506.03365*, 2015. 5

[68] Jiahui Yu, Yuanzhong Xu, Jing Yu Koh, Thang Luong, Gunjan Baid, Zirui Wang, Vijay Vasudevan, Alexander Ku, Yinfei Yang, Burcu Karagol Ayan, et al. Scaling autoregressive models for content-rich text-to-image generation. *arXiv preprint arXiv:2206.10789*, 2022. 5

[69] Qinsheng Zhang and Yongxin Chen. Fast sampling of diffusion models with exponential integrator. *arXiv preprint arXiv:2204.13902*, 2022. 2

[70] Hongkai Zheng, Weili Nie, Arash Vahdat, Kamyar Azizzadenesheli, and Anima Anandkumar. Fast sampling of diffusion models via operator learning. In *International Conference on Machine Learning*, pages 42390–42402. PMLR, 2023. 2

DeepCache: Accelerating Diffusion Models for Free

Supplementary Material

A. Pseudo algorithm

We present the pseudocode for our algorithm in Algorithm 1. It illustrates the iterative generation process over N steps, involving one step of complete model inference and N-1 steps of partial model inference. Here, we employ the sampling algorithm from DDPM [17] as an example. Our algorithm is adaptable to other fast sampling methods.

Algorithm 1: DeepCache

Input: A U-Net Model with down-sample blocks $\{D_i\}_{i=1}^{d}$, up-sample blocks$\{U_i\}_{i=1}^{d}$ and middle blocks M
Input: Caching Interval N, Branch Index m
Input: Output from step x_t, timestep t
Output: predicted output at $t-N$ step
▷ 1. Cache Step - Calculate $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ and x_{t-1}
$\mathbf{h}_0 \leftarrow \mathbf{x}_t$ ▷ $\mathbf{h}_i$ for down-sampling features
for $i = 1, \ldots, d$ **do**
  $\mathbf{h}_i \leftarrow D_i(\mathbf{h}_{i-1})$
$\mathbf{u}_{d+1} \leftarrow M(\mathbf{h}_d)$ ▷ $\mathbf{u}_i$ for up-sampling features
for $i = d, \ldots, 1$ **do**
  if $i = m$ **then**
    Store $\mathbf{u}_{i+1}$ in Cache
  $\mathbf{u}_i \leftarrow U_i(\text{Concat}(\mathbf{u}_{i+1}, \mathbf{h}_i))$
$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\mathbf{u}_1\right) + \sigma_t \mathbf{z}$ ▷ $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
▷ 2. Retrieve Step - Calculate $\{x_{t-i}\}_{i=2}^{N}$
for $n = 2, \ldots, N$ **do**
  $\mathbf{h}_0 \leftarrow \mathbf{x}_{t-n+1}$
  for $i = 1, \ldots, m$ **do**
    $\mathbf{h}_i \leftarrow D_i(\mathbf{h}_{i-1})$
  Retrieve $\mathbf{u}_{i+1}$ from Cache
  for $i = m, \ldots, 1$ **do**
    $\mathbf{u}_i \leftarrow U_i(\text{Concat}(\mathbf{u}_{i+1}, \mathbf{h}_i))$
  $\mathbf{x}_{t-n} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\mathbf{u}_1\right) + \sigma_t \mathbf{z}$ ▷ $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
return x_{t-N}

B. Varying Hyper-parameters in Non-uniform 1:N Strategy

In the non-uniform 1:N strategy, the two hyper-parameters involved are the center c and the power p, which is used to determine the sequence of timesteps for conducting the entire model inference. We test on LDM-4-G for the impact of these hyper-parameters. Results are shown in Table 7 and Table 8. From these two tables, a clear trend is evident in the observations: as the parameters p and c are incremented, there is an initial improvement in the generated image quality followed by a subsequent decline. This pattern affirms the effectiveness of the strategy and also aligns with the location of the significant decrease in similarity observed in Figure 2(c).

	ImageNet 256 × 256				
Center	**FID ↓**	**sFID ↓**	**IS ↑**	**Precision ↑**	**Recall ↑**
c = 10	8.26	8.47	160.3	75.69	48.93
c = 20	8.17	8.46	161.18	75.77	48.95
c = 50	7.77	8.16	163.74	76.23	49.18
c = 80	7.36	7.76	166.21	76.93	49.75
c = 100	7.16	7.51	167.52	77.30	49.64
c = 120	**7.11**	**7.34**	**167.85**	**77.44**	**50.08**
c = 150	7.33	7.36	166.04	77.09	49.98
c = 200	8.09	7.79	160.50	75.85	49.69

Table 7. Varying Center c with the power p equals to 1.2. Here the caching interval is set to 20.

	ImageNet 256 × 256				
Power	**FID ↓**	**sFID ↓**	**IS ↑**	**Precision ↑**	**Recall ↑**
p = 1.05	7.36	7.52	166.12	77.06	50.38
p = 1.1	7.25	7.44	166.82	77.17	50.13
p = 1.2	7.11	**7.34**	167.85	77.44	50.08
p = 1.3	**7.09**	7.35	**167.97**	**77.56**	**50.34**
p = 1.4	7.13	7.39	167.68	77.42	50.26
p = 1.5	7.25	7.44	166.82	77.17	50.13

Table 8. Varying Power p with the center c equals to 120. Here the caching interval is also set to 20.

	N=2	N=3	N=5	N=10	N=20
Center - c	120	120	110	110	120
Power - p	1.2	1.2	1.4	1.2	1.4

Table 9. Hyper-parameters for the non-uniform 1:N strategy in LDM-4-G

	N=2	N=3	N=4	N=5	N=6	N=7	N=8
Center - c	15	15	15	10	15	15	10
Power - p	1.5	1.3	1.4	1.5	1.3	1.4	1.4
Center - c	20	20	20	15	15	15	20
Power - p	1.3	1.4	1.4	1.3	1.5	1.5	1.3

Table 10. Hyper-parameters for the non-uniform 1:N strategy in Stable Diffusion v1.5.

Selected Hyper-parameters for non-uniform 1:N For experiments in LDM, the optimal hyper-parameters and shown in Table 9. For experiments in Stable Diffusion, we chose center timesteps from the set {0, 5, 10, 15, 20, 25} and power values from the set {1.1, 1.2, 1.3, 1.4, 1.5, 1.6}.

ImageNet 256 × 256 (250 DDIM Steps)											
Method	**FID ↓**	**sFID ↓**	**IS ↑**	**Precision ↑**	**Recall ↑**	**Method**	**FID ↓**	**sFID ↓**	**IS ↑**	**Precision ↑**	**Recall ↑**
Baseline - LDM-4*	3.37	5.14	204.56	82.71	53.86	Baseline - LDM-4	3.60	-	247.67	87.0	48.0
Uniform - N=2	3.39	5.11	204.09	82.75	54.07	Non-uniform - N=2	3.46	5.14	204.12	83.21	53.53
Uniform - N=3	3.44	5.11	202.79	82.65	53.81	Non-uniform - N=3	3.49	5.13	203.22	83.18	53.44
Uniform - N=5	3.59	5.16	200.45	82.36	53.31	Non-uniform - N=5	3.63	5.12	200.04	83.07	53.25
Uniform - N=10	4.41	5.57	191.11	81.26	51.53	Non-uniform - N=10	4.27	5.42	193.11	81.75	51.84
Uniform - N=20	8.23	8.08	161.83	75.31	50.57	Non-uniform - N=20	7.36	7.76	166.21	76.93	49.75

Table 11. Comparing non-uniform and uniform 1:N strategy in class-conditional generation for ImageNet using LDM-4-G. *We regenerate the images using the official checkpoint of LDM-4-G.

The optimal hyper-parameter values employed in our experiments are detailed in Table 10.

From the selected hyper-parameters, we found out that the optimal values vary slightly across different datasets. A noticeable trend is observed, indicating that the majority of optimal parameters tend to center around the 15th timestep, accompanied by a power value of approximately 1.4.

C. Non-uniform 1:N v.s. Uniform 1:N

We have shown the comparison of the non-uniform 1:N versus uniform 1:N strategy on Stable Diffusion in Figure 6. Here, we extend the comparison to ImageNet with LDM-4-G, and the corresponding results are detailed in Table 11.

In accordance with the observations from Table 11, a consistent pattern emerges compared to the findings on Stable Diffusion. Notably, when employing a substantial caching interval, the non-uniform strategy demonstrates a notable improvement, with the FID increasing from 8.23 to 7.36 with N=20. However, when dealing with a smaller caching interval (N<5), the strategy does not yield an enhancement in image quality. In fact, in certain cases, it may even lead to a slight degradation of images, as evidenced by the FID increasing from 3.39 to 3.46 for N=2.

D. Varying Skip Branches

In Table 12, we show the impact on image quality as we vary the skip branch for executing DeepCache. For our experiments, we employ the uniform 1:N strategy with N=5, and the sampling of DDIM still takes 100 steps. From the results in the table, we observe that the choice of skip branch introduces a trade-off between speed and image fidelity. Specifically, opting for the first skip branch with no down-sampling blocks and one up-sampling block yields approximately 3× acceleration, accompanied by a reduction in FID to 7.14. Additionally, certain skip branches exhibit significant performance variations, particularly the 6-th branch. The results emphasize an extra trade-off between speed and image quality, complementing the earlier noted trade-off linked to different sampling steps. This particular trade-off operates at the level of model size granularity and can be achieved without incurring additional costs.

CIFAR-10 32 × 32				
Skip Branch	**MACs ↓**	**Throughput ↑**	**Speed ↑**	**FID ↓**
1	1.60G	29.60	3.023×	7.14
2	2.24G	22.24	2.272×	5.94
3	3.01G	18.11	1.850×	5.73
4	3.89G	15.44	1.577×	5.69
5	4.58G	13.15	1.343×	5.51
6	5.31G	11.46	1.171×	4.93
7	5.45G	11.27	1.151×	4.92
8	5.60G	11.07	1.131×	4.76
9	5.88G	10.82	1.105×	4.54
10	5.95G	10.73	1.096×	4.57
11	5.99G	10.67	1.089×	4.52
12	6.03G	10.59	1.082×	4.48

Table 12. Effect of different skip branches. Here we test the impact under the uniform 1:5 strategy.

E. Prompts

Prompts in Figure 1(a):

- A bustling city street under the shine of a full moon
- A picture of a snowy mountain peak, illuminated by the first light of dawn
- dark room with volumetric light god rays shining through window onto stone fireplace in front of cloth couch
- A photo of an astronaut on a moon
- A digital illustration of a medieval town, 4k, detailed, trending in artstation, fantasy
- A photo of a cat. Focus light and create sharp, defined edges

Prompts in Figure 5:

- A person in a helmet is riding a skateboard
- There are three vases made of clay on a table
- A very thick pizza is on a plate with one piece taken.
- A bicycle is standing next to a bed in a room.
- A kitten that is sitting down by a door.
- A serene mountain landscape with a flowing river, lush greenery, and a backdrop of snow-capped peaks, in the style of an oil painting.
- A delicate floral arrangement with soft, pastel colors and light, flowing brushstrokes typical of watercolor paintings.
- A magical winter wonderland at night. Envision a land-

PLMS				DeepCache				
Steps	Throughput	Speed	CLIP Score	N	Throughput	Speed	Uniform 1:N	Non-Uniform 1:N
50	0.230	1.00	29.51	1	-	-	-	-
45	0.251	1.09	29.40	2	0.333	1.45	29.54	29.51
40	0.307	1.34	29.35	3	0.396	1.72	29.50	29.59
35	0.333	1.45	29.24	4	0.462	2.01	29.53	29.57
30	0.384	1.67	29.24	5	0.494	2.15	29.41	29.50
25	0.470	2.04	29.32	6	0.529	2.30	29.30	29.46
20	0.538	2.34	29.15	7	0.555	2.41	29.11	29.42
15	0.664	2.89	28.58	8	0.582	2.53	28.97	29.26

Table 13. Stable Diffusion v1.5 on PartiPrompt

PLMS				DeepCache				
Steps	Throughput	Speed	CLIP Score	N	Throughput	Speed	Uniform 1:N	Non-Uniform 1:N
50	0.237	1.00	30.24	1	-	-	-	-
45	0.252	1.06	30.14	2	0.356	1.50	30.31	30.37
40	0.306	1.29	30.19	3	0.397	1.68	30.33	30.34
35	0.352	1.49	30.09	4	0.448	1.89	30.28	30.31
30	0.384	1.62	30.04	5	0.500	2.11	30.19	30.23
25	0.453	1.91	29.99	6	0.524	2.21	30.04	30.18
20	0.526	2.22	29.82	7	0.555	2.34	29.90	30.10
15	0.614	2.59	29.39	8	0.583	2.46	29.76	30.02

Table 14. Stable Diffusion v1.5 on MS-COCO 2017

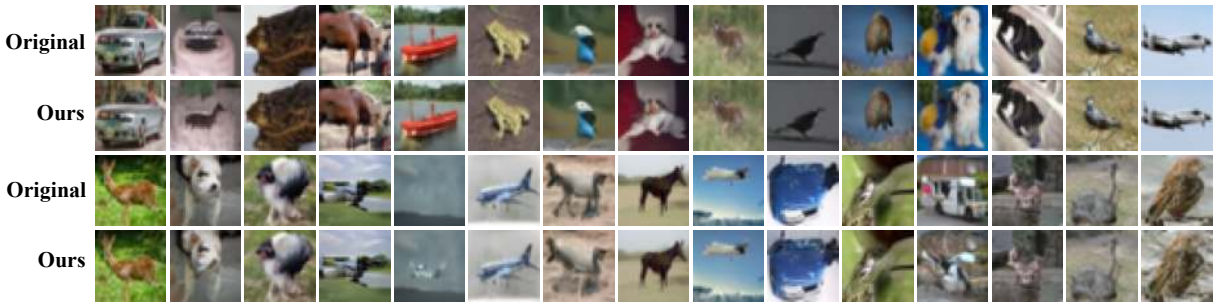

Figure 8. DDPM for LSUN-Churches: Samples with DDIM-100 steps (upper line) and DDIM-100 steps + DeepCache with N=5 (lower line). The speedup Ratio here is 1.85×.

scape covered in fresh snow, with twinkling stars above, a cozy cabin with smoke rising from its chimney, and a gentle glow from lanterns hung on the trees

- A photograph of an abandoned house at the edge of a forest, with lights mysteriously glowing from the windows, set against a backdrop of a stormy sky. high quality photography, Canon EOS R3.
- A man holding a surfboard walking on a beach next to the ocean.

F. Detailed Results for Stable Diffusion

We furnish the elaborate results corresponding to Figure 6 in Table 13 and Table 14. Given the absence of a definitive N for aligning the throughput of PLMS, we opt for an alternative approach by exploring results for various N values. Additionally, we assess the performance of the PLMS algorithm across different steps. Analyzing the data from these tables reveals that for $N < 5$, there is minimal variation in the content of the image, accompanied by only slight fluctuations in the CLIP Score.

G. More Samples for Each Dataset

We provide the generated images for each model and each dataset in Figure 8, Figure 9, Figure 10, Figure 11 and Figure 12.

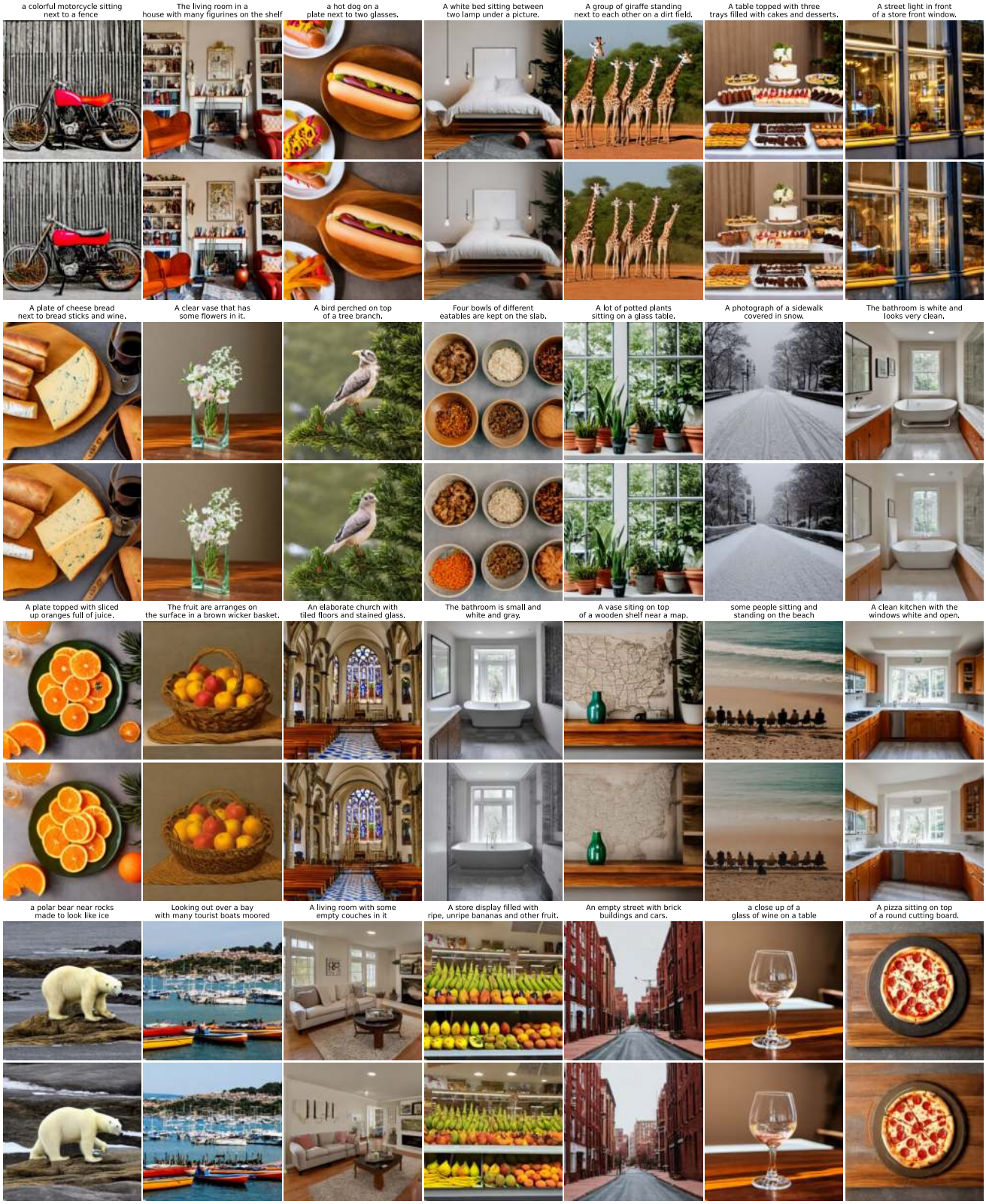

Figure 9. Stable Diffusion v1.5: Samples with 50 PLMS steps (upper line) and 50 PLMS steps + DeepCache with N=5 (lower line). The speedup Ratio here is 2.15×. Here we select prompts from the MS-COCO 2017 validation set.

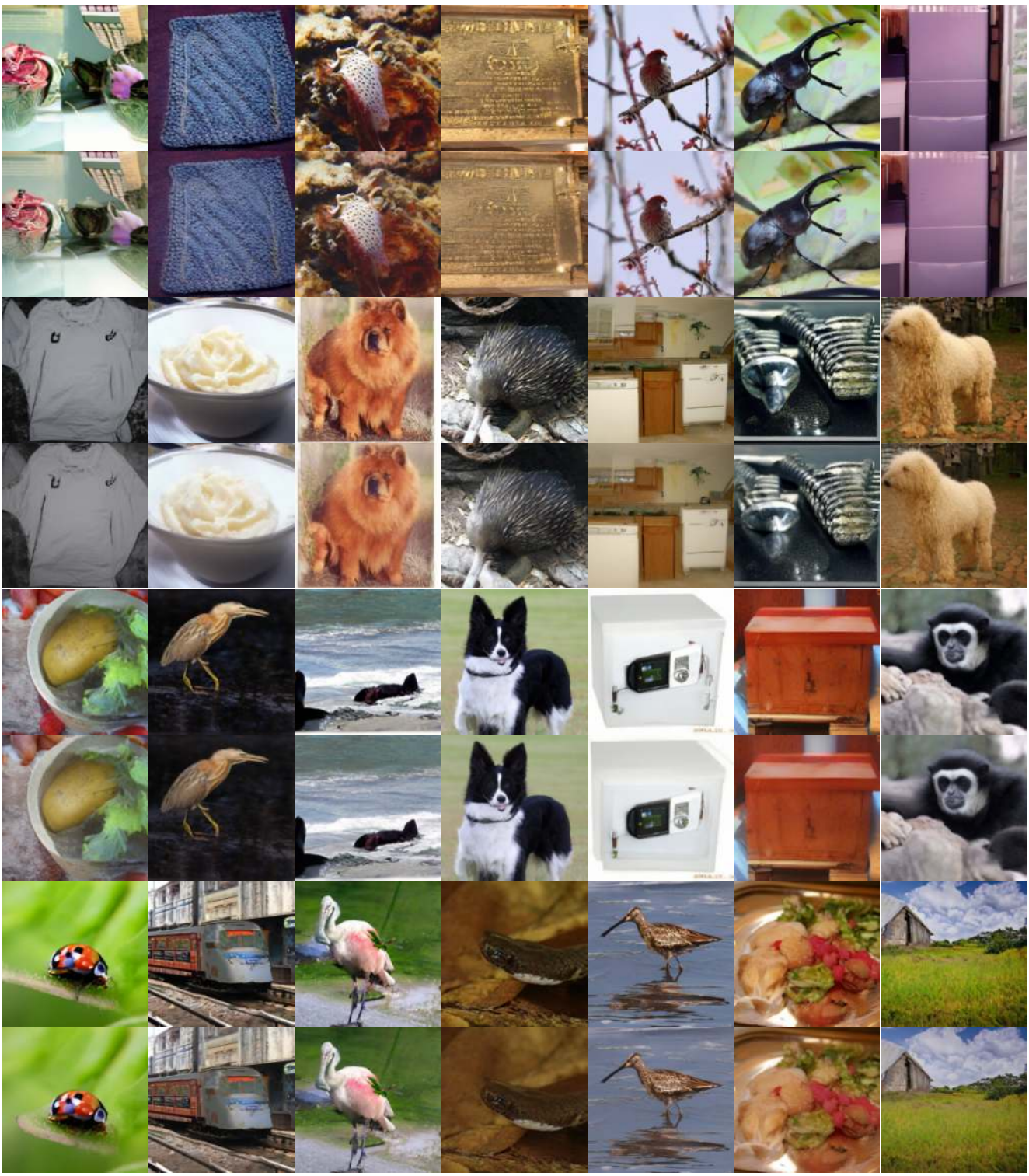

Figure 10. LDM-4-G for ImageNet: Samples with DDIM-250 steps (upper line) and DDIM-250 steps + DeepCache with N=10 (lower line). The speedup Ratio here is 6.96×.

Figure 11. DDPM for LSUN-Bedroom: Samples with DDIM-100 steps (upper line) and DDIM-100 steps + DeepCache with N=5 (lower line). The speedup Ratio here is 1.48×.

Figure 12. DDPM for LSUN-Churches: Samples with DDIM-100 steps (upper line) and DDIM-100 steps + DeepCache with N=5 (lower line). The speedup Ratio here is 1.48×.